UCS321: AI FOR ENGINEERS

MST Mini Project 1

Estimating Motor Winding Temperature

A machine learning "virtual sensor" for a Permanent Magnet Synchronous Motor

Submitted by

Name	**Roll No.**
Lakshya	_____
Atharva	_____
Vinayak Singh	_____
Mannanjit Singh	_____
Namit	_____

Submitted to: _____

Thapar Institute of Engineering & Technology, Patiala

October 2026

Abstract

Electric motors in vehicles get hot while they work, and the copper winding inside the stator is the part most at risk. Measuring its temperature directly is hard and expensive, so in this project we built a machine learning model that **estimates the winding temperature from signals that are easy to measure**: voltages, currents, motor speed, torque and the outside (ambient) temperature.

We used a public dataset of about **1.33 million measurements** from 69 test runs of a Permanent Magnet Synchronous Motor (PMSM). After exploring the data, we removed columns that would let the model "cheat" (other temperatures measured inside the motor) and we split the data **by test run**, so that the model is always judged on runs it has never seen.

We started with a simple baseline and linear models, then moved to polynomial regression, gradient boosting and a neural network. We used group-aware cross-validation and GridSearchCV to tune the models fairly. The best model, a **tuned MLP neural network, explains about 81% of the temperature variation (R^2 = 0.811) with an average error of 9.1 °C** on completely unseen runs. A much simpler Polynomial + Ridge model came close (R^2 = 0.762) and trains in seconds.

Contents

1. Introduction
2. Dataset
3. Our Approach (ML Pipeline)
4. Exploratory Data Analysis
5. Data Preprocessing
6. Choosing the Algorithms
7. How We Measure Performance
8. Experiments and Results
9. Understanding the Results
10. Conclusion, Limitations and Future Work
11. Team Contributions
12. References

1. Introduction

1.1 Background

Electric vehicles are driven by electric motors, and a very common type is the **Permanent Magnet Synchronous Motor (PMSM)**. It has a fixed outer part called the **stator**, which carries copper coils (the **winding**), and a rotating inner part called the **rotor**, which carries permanent magnets.

When current flows through the winding, it produces heat. This is called **copper loss**, and it grows with the **square of the current** (I^2R). If the winding gets too hot, the insulation on the wire breaks down and the motor can fail. If the controller is too careful, the motor is never used to its full power. So knowing the winding temperature accurately matters for both safety and performance.

1.2 Problem statement

The problem given to us (General Electric Power Systems) was to improve the reliability of electric motors by **predicting winding temperatures under different working conditions**, using parameters such as ambient temperature, motor speed, load torque, supply voltage and current, by building a **supervised machine learning regression model**.

In simple terms: a sensor on the winding is hard to fit and maintain in a real car, so we want a model that acts as a **virtual sensor**. It looks at signals the motor controller already knows and tells us how hot the winding is.

1.3 Objectives

1. Understand the data and find out which signals are related to the winding temperature.
2. Prepare the data honestly, avoiding any input that would leak the answer.
3. Build models step by step, from a simple baseline to a neural network.
4. Compare the models fairly using cross-validation and tune their settings.
5. Pick the best model and explain where it works well and where it struggles.

1.4 Type of problem

This is a **supervised regression** problem. It is supervised because every row in the data already has the real measured temperature, which the model learns from. It is regression because the answer is a number (for example 72.4 °C) and not a category like "hot" or "not hot".

2. Dataset

We used the **Electric Motor Temperature** dataset (file measures_v2.csv), recorded on a test bench by the LEA department at Paderborn University and shared publicly on Kaggle. Sensor readings were recorded **twice per second**.

Property	Value
Number of rows	1,330,816
Number of columns	13
Missing values	None
Number of test runs (profile_id)	69 (from about 2,000 to 44,000 rows each)
Target column	stator_winding (°C)

The columns are:

Column	Meaning	Unit	Used as input?
u_q, u_d	Voltages in the d-q frame	V	Yes
i_q, i_d	Currents in the d-q frame	A	Yes
motor_speed	Rotation speed	rpm	Yes
torque	Turning force	Nm	Yes
ambient	Outside air temperature	°C	Yes
coolant	Coolant temperature	°C	No
stator_tooth, stator_yoke	Other temperatures inside the stator	°C	No (leakage)
pm	Permanent magnet temperature	°C	No (leakage)
stator_winding	Winding temperature	°C	Target
profile_id	Which test run the row belongs to	-	Only for splitting

A quick note on **d-q values**: the motor is fed with three-phase AC, which is awkward to work with. Engineers convert the three phase currents into two steady numbers. **i_q** mostly creates torque, and **i_d** is mostly used to weaken the magnetic field at high speed. u_q and u_d are the matching voltages.

A **profile** is one recording session in which the motor follows a certain pattern of speed and load for a while. All the rows of one profile come one after another, so neighbouring rows are almost identical. This fact turned out to be very important for how we split the data (Section 5.2).

3. Our Approach (ML Pipeline)

We followed the standard machine learning workflow. Each step builds on the one before it, and each member of the team owned one step.

Step	What we did	Owner
1. Understand the problem	Read the problem statement and learned how a PMSM heats up	Whole team
2. Exploratory Data Analysis	Checked the data, its distributions and correlations	Lakshya
3. Preprocessing	Chose inputs, removed leaking columns, split by profile, scaled	Atharva
4. Baseline and linear models	Mean baseline, Linear, Polynomial, Ridge, Lasso	Vinayak
5. Optimisation and evaluation	Cross-validation, GridSearchCV, overfitting check, best model	Mannanjit / Vinayak
6. Interpretation and report	Plots, error analysis, conclusions, final report	Namit

Throughout the project we tried to follow one rule: **every decision should have a reason**. The rest of the report explains not only what we did, but why.

Tools used: **Python** in **Google Colab**, with **pandas** and **NumPy** for data handling, **scikit-learn** for models and evaluation, and **Matplotlib/Seaborn** for plots.

4. Exploratory Data Analysis

Before training any model we looked carefully at the data. This is like reading the whole question paper before answering.

4.1 Basic checks

- The dataset has **1,330,816 rows and 13 columns**, all numeric.
- There are **no missing values**, so no filling in (imputation) was needed.
- The 69 profiles behave very differently. For example, one profile has an average winding temperature of about 29 °C and another about 78 °C.

4.2 Distribution of the target

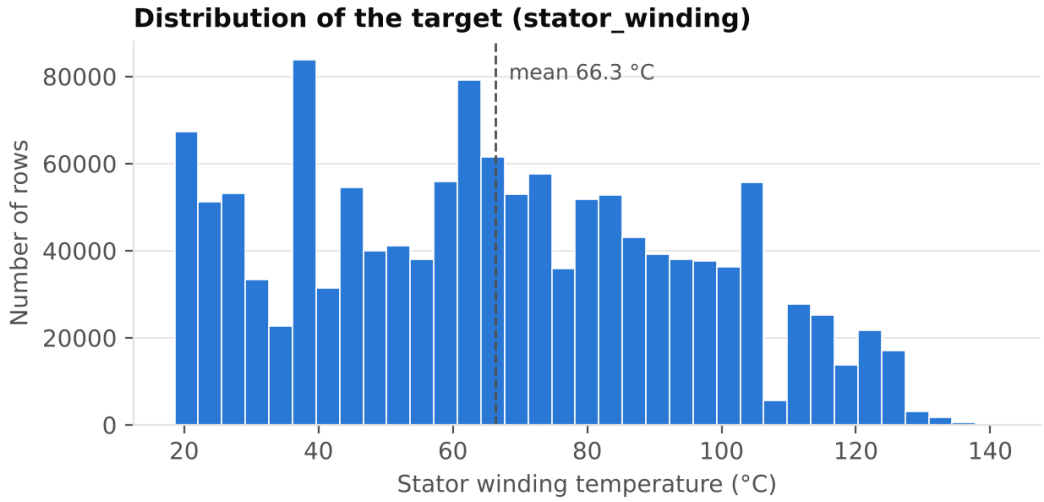


Figure 1: Distribution of the winding temperature across all rows

Most of the winding temperatures lie between about 30 °C and 100 °C, with an average of about 66 °C. Very hot readings (above 120 °C) are rare. This means the model sees fewer examples of the most dangerous cases, which we keep in mind when reading the results.

4.3 Correlation with the target

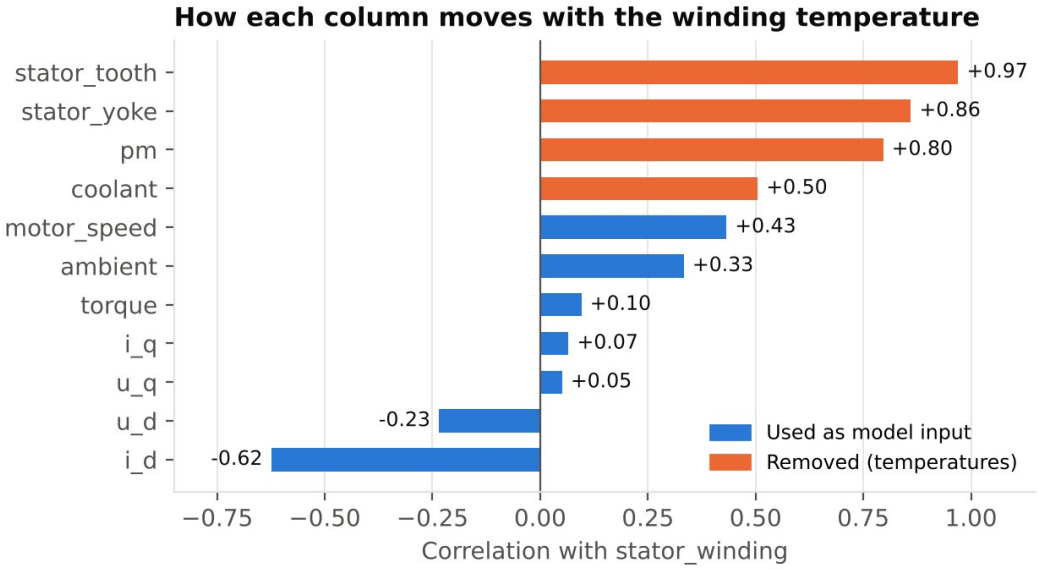


Figure 2: Correlation of each column with the winding temperature

Correlation is a number between −1 and +1 that shows how strongly two columns move together in a straight-line way. The plot shows two clear groups:

- **The other temperatures** (stator_tooth 0.97, stator_yoke 0.86, pm 0.80) move almost exactly with the winding temperature. This is not surprising, since they are measured just next to it.
- **Among the electrical and mechanical signals**, i_d has the strongest relation (−0.62), followed by motor_speed (0.43) and ambient (0.33). Torque, i_q and u_q look almost unrelated on their own.

A low correlation does **not** mean a column is useless. Correlation only measures straight-line relationships with one column at a time, while heat depends on current squared and on combinations of signals. This is a big hint that non-linear models will do better.

4.4 Engineered feature: total current

During EDA we also created a new column for the total current, **current_magnitude = $\sqrt{(i_d^2 + i_q^2)}$**. It has a correlation of about **0.57** with the winding temperature, much higher than i_q alone, which matches the physics of copper loss. Its squared version was later tested in the models (Section 8.6).

5. Data Preprocessing

5.1 Choosing the inputs (avoiding data leakage)

When we first trained a simple Ridge model using **all** the columns, it reached an R^2 of **0.997**, which is almost perfect. Looking at its coefficients showed why: it was almost entirely using **stator_tooth**. In other words, the model was just copying a thermometer placed right next to the winding.

This is called **data leakage**. In a real motor, if you could measure stator_tooth, stator_yoke or pm, you would hardly need this model. So we **removed these three columns**. We also removed **coolant**, because it is a temperature too and is not among the inputs listed in the problem statement. The final model uses **7 inputs**: u_q, u_d, i_d, i_q, motor_speed, torque and ambient.

5.2 Splitting the data by test run

Because data was recorded twice per second, two neighbouring rows of one profile are nearly identical. If we split rows randomly, row 500 could end up in training and row 501 in testing, and the model would look much better than it really is.

So we split by **profile_id** using a group-aware 80/20 split. Every profile goes **completely** into either training or testing. This way, the test score tells us how well the model works on a driving pattern it has **never seen**, which is what matters in real use. profile_id itself is not used as a model input, since it is only a label for the run.

Set	Rows	Profiles
Training	1,026,423	55
Testing	304,393	14
Profiles in both	0	0

5.3 Scaling

The inputs are on very different scales: motor speed goes up to 6,000 rpm while voltages are in the hundreds. Linear models and neural networks are sensitive to this, so we used **StandardScaler**, which shifts each column to an average of 0 and a spread of 1. The scaler is fitted on the training data only and then applied to the test data, so no information from the test set leaks into training. All scaling was done inside scikit-learn **pipelines** to keep this automatic.

6. Choosing the Algorithms

We did not jump straight to the most complex model. We started simple and added complexity only when it clearly helped, so that we could see what each step actually adds.

Model	Idea in one line	Why we tried it
Mean baseline	Always predict the average temperature	The minimum bar any model must beat
Linear Regression	Best straight-line combination of the inputs	Simple and easy to explain
Polynomial Regression	Adds squares and products of inputs (e.g. i_d^2, $i_d \times$ speed)	Heat depends on current squared
Ridge Regression	Linear regression with an L2 penalty that shrinks coefficients	Controls overfitting, needed with many polynomial terms
Lasso Regression	Linear regression with an L1 penalty that can set coefficients to zero	Shows which inputs really matter
HistGradientBoosting	Many small decision trees, each fixing the errors of the previous ones	Strong, fast model for tabular data
MLP Neural Network	Layers of neurons that can learn curved relationships	Best performer in the group's early benchmark

Before settling on these, the group ran a quick benchmark of many algorithms from the syllabus (Decision Tree, Random Forest, KNN, SVR, AdaBoost, XGBoost and others) on a smaller sample. That benchmark is shown in Section 8.2.

7. How We Measure Performance

We used three standard regression metrics, always calculated on data the model did not train on.

Metric	What it means in simple words	Better when
MAE (Mean Absolute Error)	The average size of the mistake, in °C	Lower
RMSE (Root Mean Squared Error)	Like MAE, but large mistakes count more. Always at least as big as MAE	Lower
R^2 (coefficient of determination)	Share of the temperature variation the model explains. 1 is perfect, 0 is no better than the average	Higher

For example, if the real temperatures are 50, 60 and 70 °C and the model predicts 52, 57 and 70 °C, the errors are 2, 3 and 0, so the MAE is 1.67 °C and the RMSE is 2.08 °C. RMSE is higher because the 3 °C error counts extra.

We also used two evaluation techniques:

- **Cross-validation with GroupKFold (5 folds)**: the training data is split into 5 groups of profiles; each model is trained 5 times, each time tested on a different group, and the scores are averaged. This gives a more reliable estimate than a single split.
- **Train vs test comparison**: if the training score is much higher than the test score, the model is **overfitting** (memorising). If both are low, it is **underfitting** (too simple).

8. Experiments and Results

8.1 Baseline and linear models

All models were trained on the full training set (55 profiles) and tested on the 14 unseen profiles.

Model	Test MAE (°C)	Test RMSE (°C)	Test R^2	Train R^2
Mean baseline	25.90	30.35	0.000	-
Linear Regression	16.45	19.93	0.568	0.495
Ridge Regression	16.45	19.93	0.568	0.495
Lasso Regression	16.59	20.01	0.565	0.493
Polynomial Regression (degree 2)	**11.11**	**14.63**	**0.768**	**0.694**

- Every model clearly beats the baseline, so the inputs really do carry information about the winding temperature.
- Linear, Ridge and Lasso give almost the same score. With over a million training rows, the plain linear model is already stable and the penalty has little to correct.
- Lasso set the coefficients of i_q, motor_speed and torque to zero. Torque is produced mainly by i_q, so these columns repeat the same information.
- **Polynomial regression jumps from 0.57 to 0.77.** Adding squared terms such as i_d^2 lets the model describe copper loss, which grows with current squared.
- The training scores are not higher than the test scores, so these models are **underfitting**: they are simply too simple for this problem.

8.2 Early benchmark of syllabus models

To decide which advanced models were worth tuning, the group first trained many algorithms on a smaller sample (55,000 training rows, 30,000 test rows from the unseen profiles). These numbers are not directly comparable with the full-data results, but they show which families are promising.

Model	MAE (°C)	R^2
MLP Neural Network	**10.32**	**0.779**
SVR (RBF kernel, 12,000 rows)	9.78	0.767
XGBoost	14.25	0.632
Extra Trees	14.52	0.622
Gradient Boosting	14.87	0.612
Random Forest	15.24	0.561
AdaBoost	16.50	0.560
KNN (12,000 rows)	15.51	0.538
Decision Tree	16.26	0.479

The MLP and SVR stood out. SVR gets very slow as the data grows, so it could only be trained on 12,000 rows, while the MLP can use all one million rows. We therefore carried the **MLP**, a fast boosting model (**HistGradientBoosting**) and the best linear-family model (**Polynomial + Ridge**) forward to tuning.

8.3 Cross-validation (default settings)

Each model was evaluated with 5-fold GroupKFold on a sample of about 1,000 rows from each training profile (52,868 rows), to keep the run time reasonable.

Model	CV MAE (°C)	CV R^2	Spread of R^2 (std)
MLP Neural Network	**14.46**	**0.506**	**0.043**
HistGradientBoosting	15.24	0.490	0.088
Polynomial (degree 2)	13.18	0.413	0.364
Linear / Ridge	17.36	0.372	0.116
Lasso	17.45	0.371	0.116

The MLP had the best and also the **most stable** score across folds. Polynomial regression looked fine on average but its score swung a lot from fold to fold (spread 0.36), meaning it does very well on some runs and badly on others. Cross-validation scores are lower than the final test scores because they use a smaller sample and some training profiles are harder to predict than the test profiles. The ranking of the models, which is what we needed, stayed the same.

8.4 Hyperparameter tuning with GridSearchCV

A **hyperparameter** is a setting we choose before training, such as the number of layers in a neural network. **GridSearchCV** tries every combination from a list, scores each with 5-fold GroupKFold, and keeps the best one.

Model	Settings tried	Best setting found
Polynomial + Ridge	degree 1, 2, 3; alpha 0.1, 1, 10, 100	degree 3, alpha 1
HistGradientBoosting	learning rate 0.05, 0.1; leaf nodes 15, 31; L2 0, 1	learning rate 0.05, 15 leaves, L2 = 1
MLP Neural Network	hidden layers (32), (64, 32), (128, 64); alpha 0.0001, 0.01	(128, 64), alpha 0.01

8.5 Final test on unseen profiles

Each tuned model was retrained on all 1,026,423 training rows and tested **once** on the 14 held-out profiles.

Tuned model	Train R^2	Test R^2	Gap	Test MAE (°C)	Test RMSE (°C)
MLP Neural Network	**0.869**	**0.811**	**0.058**	**9.07**	**13.19**
Polynomial (degree 3) + Ridge	0.769	0.762	0.008	9.80	14.82
HistGradientBoosting	0.728	0.637	0.091	14.20	18.27

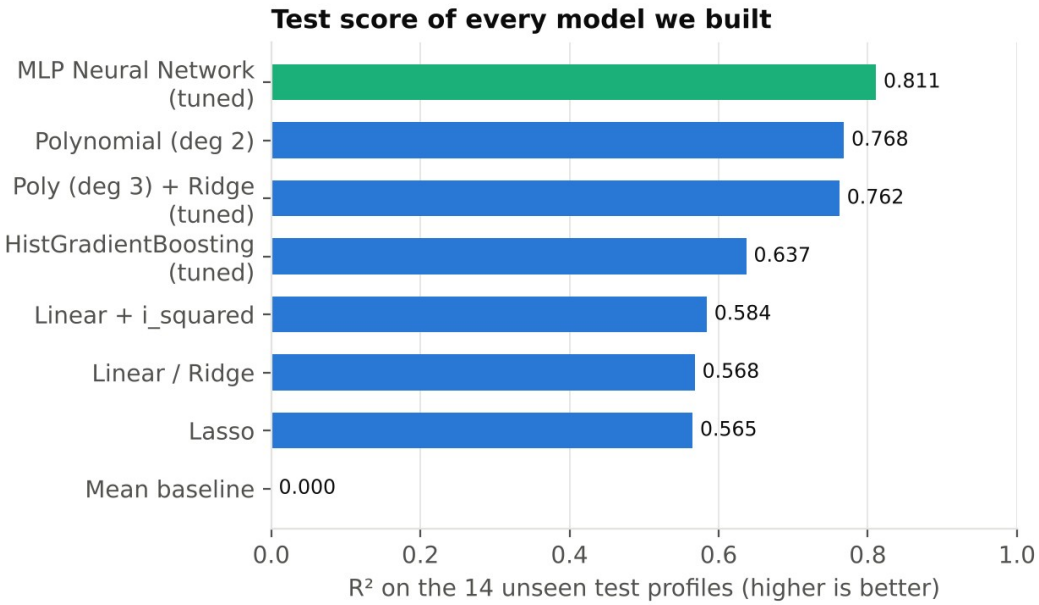


Figure 3: Test R^2 of every model, from the baseline to the tuned neural network

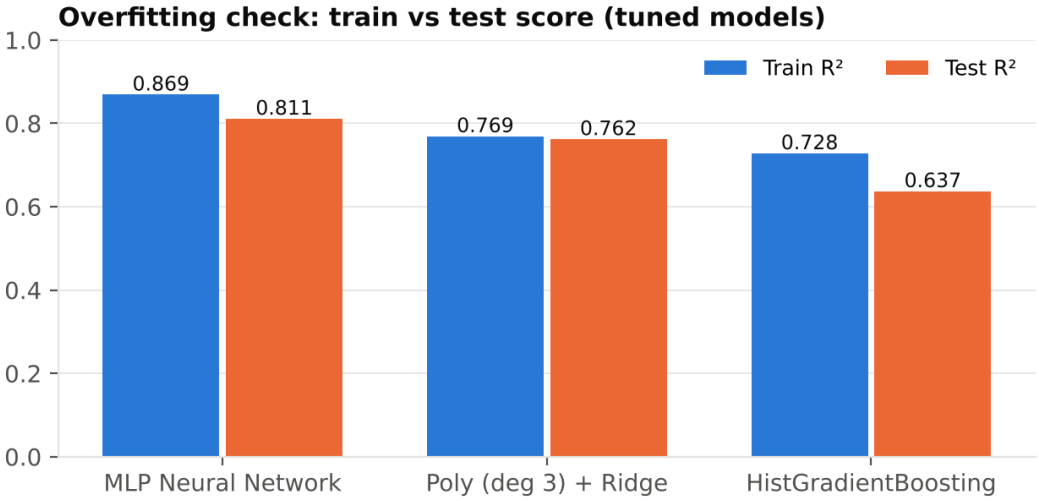


Figure 4: Training vs test score of the tuned models

- **The tuned MLP is the best model**, with $R^2 = 0.811$ and MAE = 9.07 °C on runs it has never seen. Tuning made a clear difference: the group's earlier untuned MLP reached 0.74 on the same test runs.
- **Polynomial + Ridge** comes surprisingly close (0.762). It trains in about 8 seconds, compared with over an hour for the MLP on our machine, and it has almost no gap between training and test.
- **HistGradientBoosting** has the largest gap (0.091), so it overfits the training profiles the most.

8.6 Extra test: the engineered feature i_squared

Since copper loss depends on current squared, we tested an extra input **$i_squared = i_d^2 + i_q^2$** in the linear-family models.

Model	R^2 (7 inputs)	R^2 (+ i_squared)	MAE 7 → 8 inputs (°C)
Linear / Ridge	0.568	0.584	16.45 → 16.05
Lasso	0.565	0.583	16.59 → 16.20
Polynomial (degree 2)	0.768	0.759	11.11 → 11.26
Polynomial (degree 3) + Ridge	0.762	0.759	9.80 → 9.65

The straight-line models improved a little, because they cannot create squared terms on their own. With the new column, Lasso gave i_squared one of its largest weights and dropped i_d and torque, preferring total current squared over the separate currents. The polynomial models did not improve, since they already contain i_d^2 and i_q^2. Because the gain was small and only for the simplest models, the final models keep the original 7 inputs.

9. Understanding the Results

9.1 Actual vs predicted

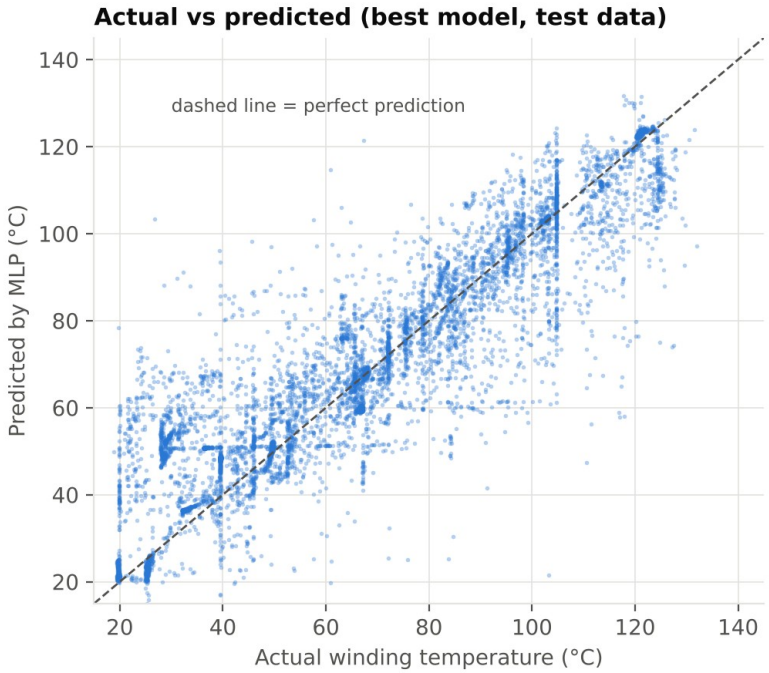


Figure 5: Actual vs predicted temperature for the best model (8,000 random test rows)

If the model were perfect, every point would lie on the dashed line. Most points sit close to it across the whole temperature range. The cloud is wider at high temperatures, which matches what we saw in EDA: hot cases are rarer, so the model has had less practice with them.

9.2 Errors (residuals)

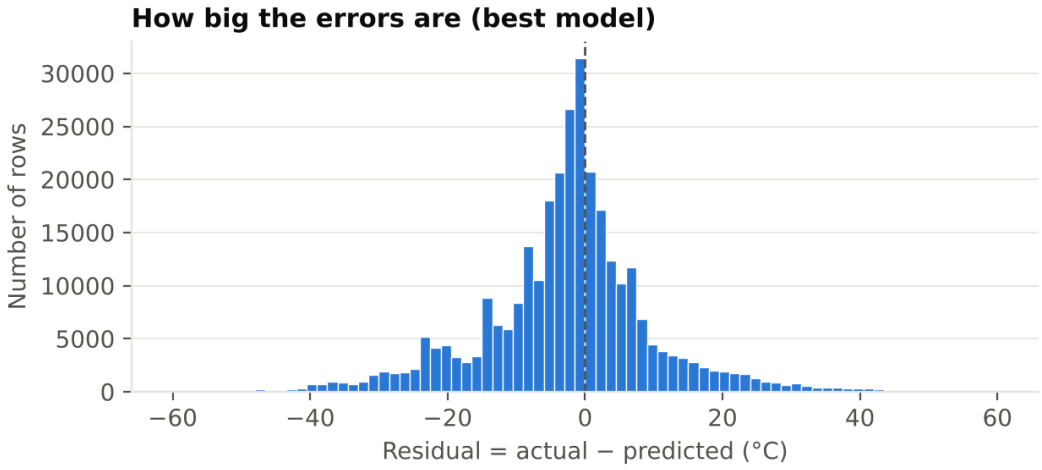


Figure 6: Distribution of the errors of the best model

The errors are centred close to zero, so the model is not strongly biased in one direction. On average it predicts about 2.5 °C higher than the real value, which is the safer side for motor

protection. About 46% of all test predictions are within 5 °C of the real temperature, 69% within 10 °C and 86% within 20 °C.

9.3 One test run over time

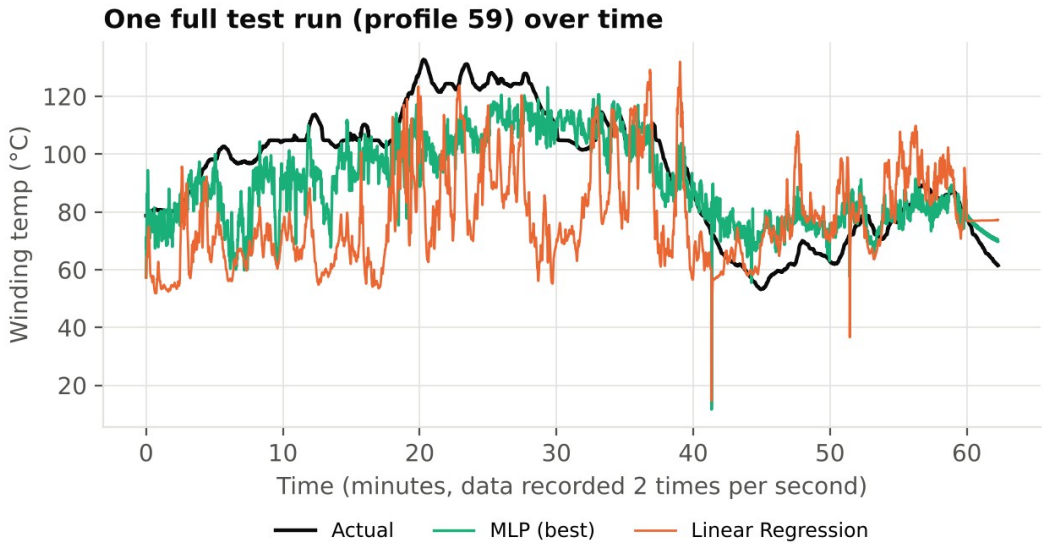


Figure 7: Actual temperature and predictions over one complete unseen test run

This plot shows profile 59, a full one-hour test run that the models never saw (its error is close to the middle of all test runs). The MLP follows the overall rise and fall of the temperature much better than Linear Regression, which often misses the shape by 30 °C or more. The MLP line is also quite jumpy, while the real temperature changes smoothly. This is because the model looks at one moment at a time and has no memory of the last few minutes, which is one of the main limitations we discuss in Section 10.

9.4 Error on each test profile

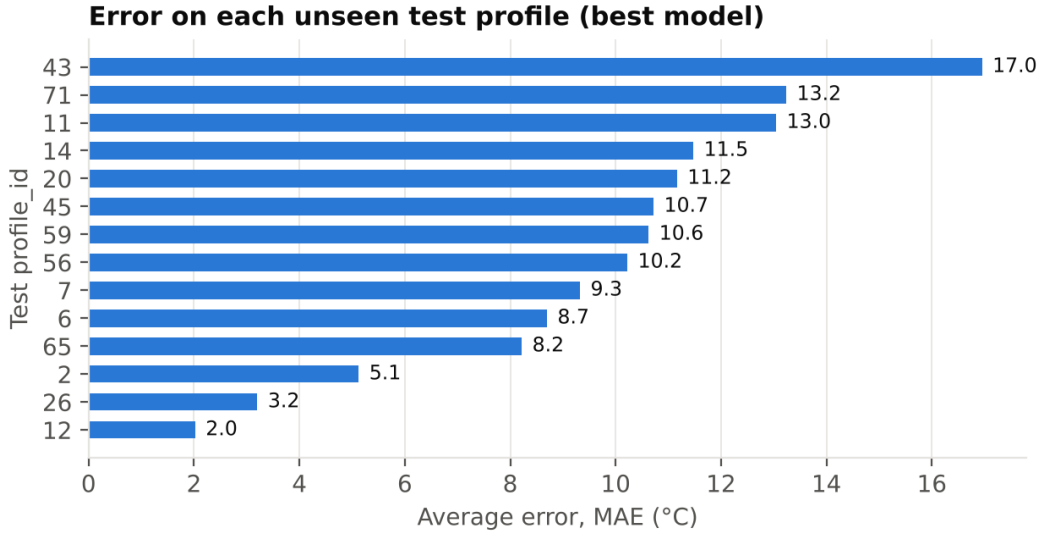


Figure 8: Average error of the best model on each of the 14 unseen profiles

The error changes a lot from one driving pattern to another. Profiles 12 and 26 are predicted within about 2 to 3 °C on average, while profile 43 is off by about 17 °C. So the overall average error of 9.1 °C hides a few difficult runs. Looking at why these runs are hard (for example long periods of heavy load) would be a good next step.

9.5 Which inputs matter most

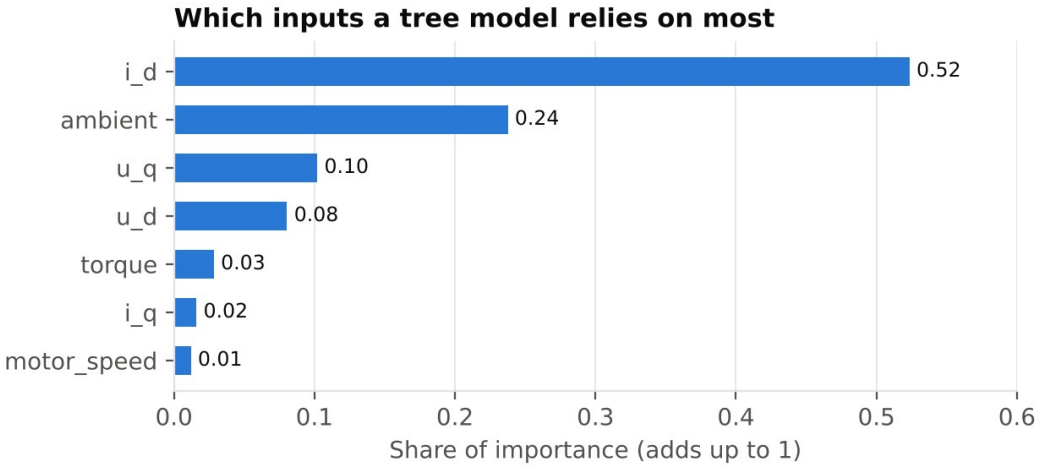


Figure 9: Feature importance from the group's gradient boosting model (7 inputs, full data)

Feature importance shows how much each input helps the model. In the group's gradient boosting model, **i_d** was by far the most important input (52%), followed by **ambient** temperature (24%) and the two voltages. This agrees with the EDA, where i_d had the strongest correlation with the winding temperature, and with the physics: the current that flows through the winding and the starting temperature of the surroundings decide how hot it gets. Torque, i_q and speed added little on their own because their information overlaps with the other inputs.

10. Conclusion, Limitations and Future Work

10.1 Conclusion

We built a machine learning virtual sensor that estimates the stator winding temperature of a PMSM from voltages, currents, speed, torque and ambient temperature. By removing leaking temperature columns and splitting the data by test run, we made sure our results reflect how the model would behave on new driving patterns.

Simple straight-line models explain only about 57% of the variation. Adding squared terms, which matches the physics of copper loss, lifts this to about 77%. After fair cross-validation and tuning, the **MLP neural network is our final model with $R^2 = 0.811$, MAE = 9.07 °C and RMSE = 13.19 °C** on 14 unseen profiles. If training speed or explainability matters more, **Polynomial (degree 3) + Ridge** is a strong alternative ($R^2 = 0.762$).

10.2 Limitations

- **No memory of the past.** The model looks at one moment at a time, but a motor heats up slowly. It does not know whether the motor has been working hard for the last ten minutes.
- **Uneven accuracy.** Some driving patterns are predicted very well and others much worse.
- **Fewer hot examples.** Very high temperatures are rare in the data, and these are the most important cases for safety.
- **Training cost.** The MLP takes much longer to train than the polynomial model.

10.3 Future work

- Add time-history features, such as moving averages of current and speed over the last few minutes, so the model can "remember" recent load.
- Try models designed for sequences, such as recurrent neural networks.
- Give more weight to high-temperature cases, since under-predicting a hot winding is the most dangerous mistake.

11. Team Contributions

Member	Role	Main contributions
Lakshya	Person 1: EDA	Dataset overview, missing-value check, distributions, correlation analysis, EDA conclusions
Atharva	Person 2: Preprocessing	Feature selection, removal of leaking columns, scaling, train/test preparation
Vinayak Singh	Person 3: Baseline & linear models (+ completed Person 4)	Mean baseline, Linear, Polynomial, Ridge and Lasso models; GroupKFold cross-validation; GridSearchCV tuning; overfitting check; engineered feature test; final model comparison
Mannanjit Singh	Person 4: Optimisation & evaluation (initial)	Early EDA and leakage analysis, group-aware split, benchmark of syllabus models, first full-data MLP and gradient boosting runs, team coordination
Namit	Person 5: Interpretation & integration	Result interpretation, plots, final compilation of report and presentation

Every member is expected to understand the full pipeline, not just their own part, so that any of us can explain any section during the viva.

12. References

1. Kirchgässner, W., Wallscheid, O., and Böcker, J. "Electric Motor Temperature" dataset, Paderborn University (LEA). Available on Kaggle.
2. Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825–2830, 2011.
3. scikit-learn documentation: GroupKFold, GroupShuffleSplit, GridSearchCV, PolynomialFeatures, Ridge, Lasso, MLPRegressor, HistGradientBoostingRegressor.
4. McKinney, W. "Data Structures for Statistical Computing in Python." Proceedings of the 9th Python in Science Conference, 2010 (pandas).
5. Hunter, J. D. "Matplotlib: A 2D Graphics Environment." Computing in Science & Engineering, 9(3), 90–95, 2007.
6. UCS321 AI for Engineers course material and MST mini project guidelines, Thapar Institute of Engineering & Technology.